AEGIS HELPDESK

Project Setup & Execution Guide

Enterprise-Ready | AI-Augmented | Event-Driven Ticketing

Version 1.0

Table of Contents

Table of Contents 2
1. Architecture & Technology Stack 3
2. Prerequisites & Database Setup 3
  2.1 System Requirements 3
  2.2 Database Initialisation 3
3. Backend Configuration (.env) 4
  3.1 Core Environment Variables 4
  3.2 File Storage — Azure Blob Storage vs. Local Fallback 4
  3.3 Text Embeddings — Tiered Provider Selection 4
    Tier 1 — Azure OpenAI (Recommended for Production) 4
    Tier 2 — Standard OpenAI 5
    Tier 3 — Deterministic Mock Embeddings (Free / Offline) 5
  3.4 Email (SMTP) 5
4. Running the Backend 5
5. Running the React Frontend 5
6. Running the Vanilla Prototype (mock-ui) 6
7. Default Login Credentials 6
8. Troubleshooting 6
  Timezone Conflict 6
  Database Driver / Connection Error 6
  401 Unauthorized Errors 7
Appendix — Port Reference 7

1. Architecture & Technology Stack

Aegis Helpdesk follows a decoupled client-server architecture comprising four modules. The table below summarises each module's technology choices and primary role within the system.

Module	Technology	Purpose
helpdesk-backend	Spring Boot, JPA, Flyway, PostgreSQL	REST & WebSocket API, DB migrations, AI integration
helpdesk-backend	Groq LLM, Azure OpenAI / OpenAI	Chat completions, ticket summaries, KB generation
helpdesk-react-frontend	React, Vite, Tailwind CSS	Modern agent & admin dashboard
mock-ui	Vanilla JS, localStorage	Client-side prototype — no backend required
sample-ui	Static assets	UI screenshots and design reference slices

> **NOTE** The mock-ui module runs entirely in the browser using localStorage as a simulated database. No backend setup is required to explore UI flows using this module.

2. Prerequisites & Database Setup

2.1 System Requirements

Ensure the following software is installed on your system before proceeding:

- Java 17 JDK (Eclipse Temurin or OpenJDK 17 recommended)
- Node.js v18.0.0 or higher
- PostgreSQL v15.0 or higher with the pgvector extension available in the server's library path

2.2 Database Initialisation

1. Open your PostgreSQL client (psql CLI or pgAdmin).
2. Create the target database:

```
CREATE DATABASE helpdesk;
```

3. Verify that your PostgreSQL user has sufficient privileges to install extensions. The Flyway migration scripts will automatically execute CREATE EXTENSION IF NOT EXISTS vector; on startup, but the pgvector shared library must already be present in PostgreSQL's library path.
4. Configure the database connection parameters in the .env file (see Section 3).

> **NOTE** The default PostgreSQL port is 5432. If your instance is not running on port 5432, update the DB_URL variable in .env accordingly. Do not confuse this with the example port 5433 shown in some configuration comments.

3. Backend Configuration (.env)

The backend reads all runtime settings from a .env file located in the root of the helpdesk-backend directory. Copy the provided example and edit as required:

```
cd helpdesk-backend
copy .env.example .env        # Windows
cp  .env.example .env          # macOS / Linux
```

3.1 Core Environment Variables

Variable	Description
DB_URL	JDBC connection string for the main PostgreSQL database. Default: jdbc:postgresql://localhost:5432/helpdesk?options=-c%20TimeZone=UTC
DB_USERNAME	PostgreSQL username. Default: postgres
DB_PASSWORD	PostgreSQL password. Default: 1234
DB_TEST_URL	(Optional) JDBC connection string for the test database used by the JUnit suite. Default: port 5432, database helpdesk_test
DB_TEST_USERNAME	(Optional) Test database username.
DB_TEST_PASSWORD	(Optional) Test database password.
JWT_SECRET	A 256-bit Base64 string used to sign user session tokens. Must be kept secret in production.
GROQ_API_KEY	Required for all AI features (chat completions, ticket summaries, draft replies, KB generation). Obtain a free key at console.groq.com.

3.2 File Storage — Azure Blob Storage vs. Local Fallback

Ticket attachments can be stored in Azure Blob Storage or on the local filesystem:

- Azure Blob Storage (default if configured): Provide a valid AZURE_STORAGE_CONNECTION_STRING in .env.
- Local fallback: Leave AZURE_STORAGE_CONNECTION_STRING blank or omit the line entirely. The application will log a warning and save attachments to the /uploads/ folder inside the project directory. These files are served publicly on port 8080.

3.3 Text Embeddings — Tiered Provider Selection

Text embeddings power duplicate ticket detection and semantic knowledge-base search. The application selects a provider automatically based on which API keys are supplied in .env:

Tier 1 — Azure OpenAI (Recommended for Production)

Supply all four Azure variables:

- AZURE_OPENAI_ENDPOINT
- AZURE_OPENAI_API_KEY
- AZURE_OPENAI_EMBEDDING_DEPLOYMENT
- AZURE_OPENAI_API_VERSION

Tier 2 — Standard OpenAI

If Azure OpenAI credentials are absent, provide OPENAI_API_KEY (e.g. sk-proj-...). The application will use the text-embedding-3-small model via the standard OpenAI /embeddings endpoint.

Tier 3 — Deterministic Mock Embeddings (Free / Offline)

If neither Azure nor Standard OpenAI keys are provided, the application falls back to a local mathematical mock generator. This produces a 1536-dimensional unit vector seeded from the hash of the input text. All backend compilation, database operations, and ticketing flows function normally with no paid AI subscriptions required.

> **NOTE** Mock embeddings are suitable for development and UI demonstrations only. Semantic similarity results will not reflect genuine language understanding.

3.4 Email (SMTP)

Transactional notifications are sent via SMTP. The default configuration targets Gmail's SMTP service. Adjust spring.mail.* properties in application.properties to point to your preferred SMTP server, or disable email features if not required.

4. Running the Backend

5. Open a terminal inside the helpdesk-backend directory.
6. Compile and start the application using the Maven wrapper:

```
# Windows
.\mvnw.cmd spring-boot:run

# macOS / Linux
./mvnw spring-boot:run
```

7. The server will start, execute Flyway schema migrations, and begin listening on http://localhost:8080.

5. Running the React Frontend

8. Open a terminal inside the helpdesk-react-frontend directory.
9. Install Node.js dependencies:

```
npm install
```

10. Launch the Vite development server:

```
npm run dev
```

11. Open the URL displayed in the console in your web browser (typically http://localhost:5173).

> **NOTE** Vite is pre-configured in vite.config.js to proxy all /api, /ws, and /uploads requests to the backend at http://localhost:8080. No manual CORS configuration is required.

6. Running the Vanilla Prototype (mock-ui)

The mock-ui module provides a fully client-side prototype suitable for rapid UI demonstrations. No database, Java runtime, or Node.js installation is needed.

12. Navigate to the mock-ui directory.
13. Double-click index.html to open it in any modern web browser.

The interface uses browser localStorage to simulate ticket creation, agent replies, and AI auto-deflection flows.

7. Default Login Credentials

When the backend starts for the first time it seeds the database with the following test accounts. Use these to explore different workspace perspectives:

Role	Username / Email	Password	Description
User (Customer)	nilesh@helpdesk.com	password123	Can submit tickets, view tickets, rate resolved tickets, search KB
Agent	sarah@helpdesk.com	password123	Receives auto-assigned tickets, can use AI copilot, draft replies
Admin	admin@helpdesk.com	password123	Can manage categories, create agents, view metrics & analytics

8. Troubleshooting

Timezone Conflict

If PostgreSQL logs a fatal error related to Asia/Calcutta, align the timezone in the datasource URL with your PostgreSQL server locale (e.g. UTC, IST, or Asia/Kolkata).

Database Driver / Connection Error

Confirm that the PostgreSQL service is running and accessible on the port specified in DB_URL. Check firewall rules and authentication settings (pg_hba.conf) if connectivity fails.

401 Unauthorized Errors

User sessions expire according to the app.jwt.expiration setting. Clear browser localStorage or log out and log back in to obtain a fresh JWT token.

Appendix — Port Reference

Variable	Description
8080	Spring Boot backend (REST API, WebSocket, static file serving for /uploads)
5173	Vite development server (React frontend)
5432	PostgreSQL (default port — update if your instance differs)